Guidelines

- Kindly use the given template for submitting your prototype(Make a copy of the template)
- One team is only required to submit one project.
- Follow this template while preparing your presentation.
- You are welcome to add as many POCs and design concepts to support your project.
- The project should be feasible and the team members should be capable enough, to come up with the fully functioning prototype of the same idea, if required.

Team Details

a. Team name: <<< FILL IN >>>
b. Team leader name: <<< FILL IN >>>

Brief about your solution

CALIPER turns a messy supplier row into Unilog's **252-column delivery format** — and can show you why it wrote every single cell.

Most pipelines hand the row to a language model and validate what comes back. CALIPER makes that structurally impossible. Every extractor — rules, the brand registry, the taxonomy, family consensus and the model — writes into one typed **Product Fact Graph, where a fact without evidence is rejected at insertion**. Composition, validation and export only ever read from it. The model is never permitted to write an output cell: it may propose a fact that quotes the source, or judge a fact that already exists.

1,000 → 252
rows to columns, in ~3 seconds

100%
INVOICE_DESC within 40 characters

99.5%
sibling agreement, 1,516 comparisons

26,596
cells carrying their own evidence

Runs with no API key and no third-party packages. A judge opens the link and 1,000 real industrial SKUs are already enriched; clicking any row reveals the rule that produced each value and the exact characters of the input that justified it.

1. How does your solution enrich minimal product information?

Input is six columns — part number, description, three inconsistent brand fields and a manufacturer string. From that, CALIPER derives facts and never invents them:

Parse, don't guess. Dimensions, grits, wattages, counts and materials are lifted from the description by typed rules. Each becomes a fact carrying the exact substring it came from: a description reading 6 in x .045 in x 7/8 in yields three separate measurements, each traceable.

Resolve identity against a registry. "Milw" becomes Milwaukee®; distributor noise like "(4031)" is stripped. Where brand and manufacturer disagree, that is reported as a defect in the supplier's data rather than silently overwritten.

Borrow from siblings, with corroboration. Products are clustered into families by part-number and description shape. A fact missing on one row is filled only when at least two agreeing siblings supply it — and disagreement lowers confidence.

Induce the category rulebook from the data. Which attributes a category has, and which are filterable, is derived from the rows themselves with a fill rate behind every attribute — the work normally done by hand for tens of thousands of categories.

Compose to the format, not to a prompt. Five descriptions are generated from the same facts under their own length and casing rules, so they cannot contradict each other.

Opportunities

a. How different is it from any of the other existing ideas?
b. How will it be able to solve the problem statement given?
c. USP of the proposed solution

How different is it from existing ideas?

Every other approach we found prompts a model and validates afterwards, so a wrong value has already been written. CALIPER inverts the control flow: the model cannot reach an output cell at all. It proposes facts that must quote the source, or audits facts that already exist — and an unquoted proposal is discarded, not corrected.

How does it solve the problem statement?

It produces all 252 delivery columns from six, at ~3 seconds per 1,000 rows, with 77.4% of rows ready to publish unattended and the remainder routed to a review queue ranked by how much that review is worth. It also reports defects it finds in the supplier's own catalogue.

USP of the proposed solution

Auditability as an architectural guarantee, not a report. 26,596 populated cells each carry a rule id, a confidence and the source characters behind them. Because two labelled rows cannot support a claim, we also publish a label-free measure — sibling agreement across all 1,000 rows, 99.5% over 1,516 comparisons.

List of features offered by the solution

Evidence panel
Click a row: every populated column shows its method, rule id, confidence and the source substring.

Character-budget solver
INVOICE_DESC's 40-character upper-case ceiling solved as a budget problem. 100% compliant by construction.

Schema detection
Column roles are detected, not assumed — the same pipeline runs on a file whose headers it has never seen.

Category-spec induction
Unilog-style category rulebooks derived from the rows, with a fill rate behind every attribute.

Family consensus + anomaly flags
Gaps filled only from corroborating siblings; rows that break their family pattern are surfaced.

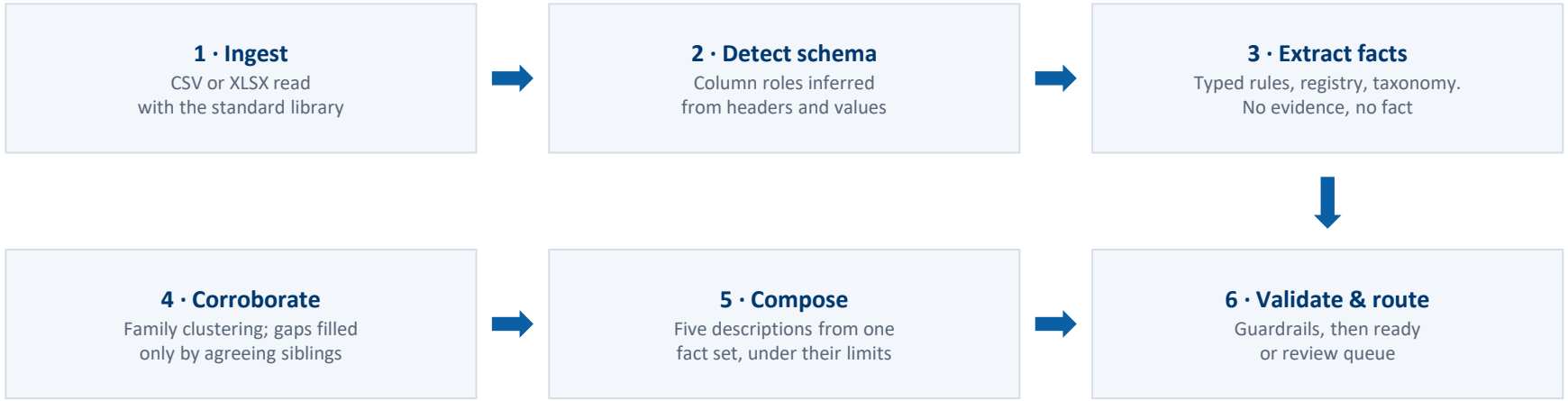
Physical guardrails
Domain ranges and dimensional coherence. These caught a bug in our own parser.

Findings on the source data
Brand/manufacturer conflicts and unresolvable rows reported rather than absorbed.

Review queue
The rows a human should look at, ranked by the value of looking.

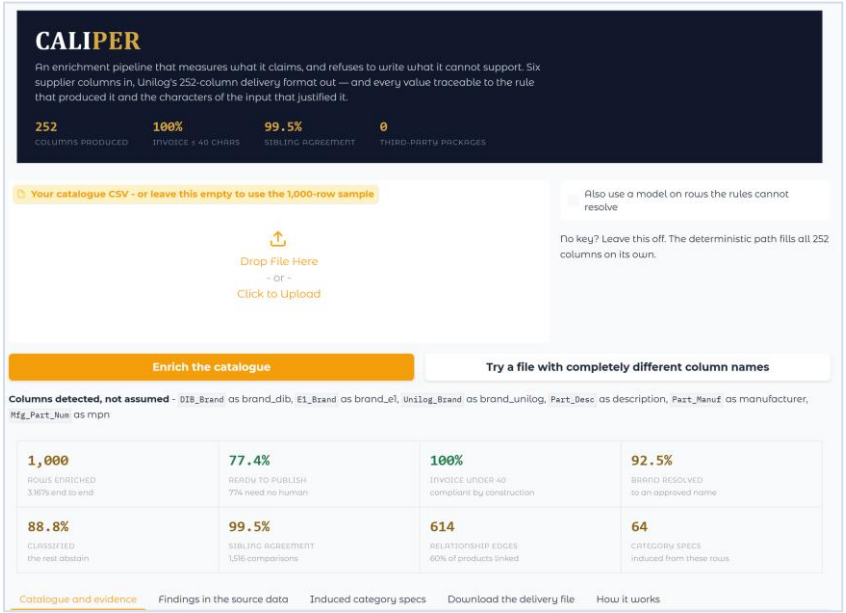
Optional model, firewalled
Bring your own key for extra recall. Held in memory only; proposals without a verbatim quote are discarded.

Process flow diagram or Use-case diagram



The one-way rule: steps 1–4 write facts into the graph. Steps 5–6 only read from it. A model may join step 3 as a proposer whose quote is checked against the source, or step 6 as an auditor — it is never given write access to an output column.

Wireframes/Mock diagrams of the proposed solution (optional)



Screenshot of the running prototype, light theme; the page also renders correctly in dark mode.

The interface, in the order a judge meets it

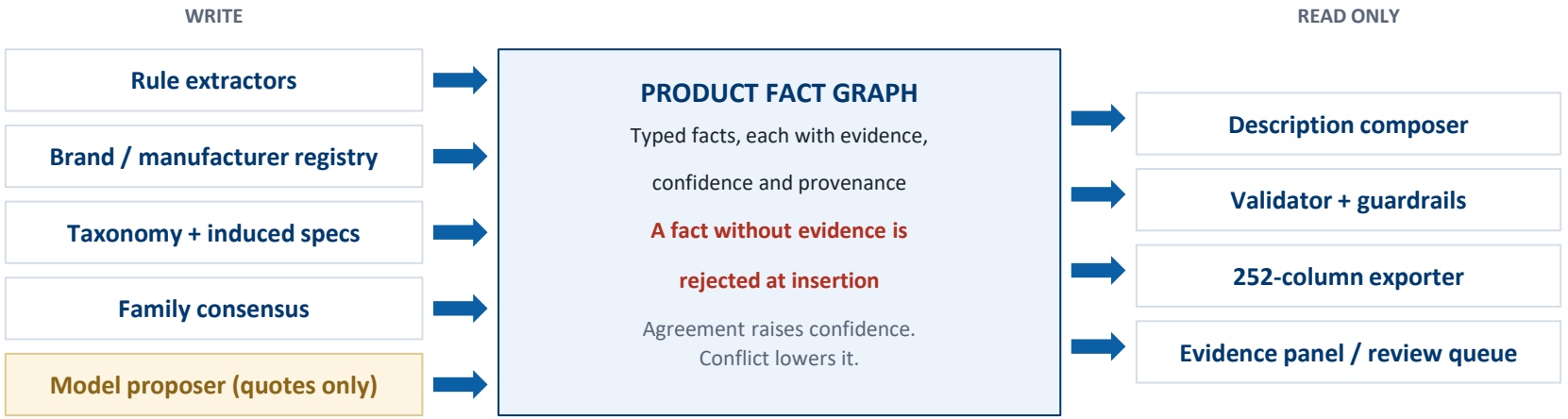
It is already running. The page enriches 1,000 SKUs on load. No button, no key, no setup — the first ten seconds show output, not a form.

Upload, or use the sample. Drop in any catalogue CSV. Column roles are detected and printed back.

The model is opt-in. Ticking the box reveals a provider and a key field. Left alone, the deterministic path still fills all 252 columns.

Then: catalogue, evidence, findings, induced specs, downloads. Five tabs, in that order — the evidence panel sits directly under the table so one click explains a row.

Architecture diagram of the proposed solution



The boundary is one-way and that is the whole design. Readers cannot create values; writers cannot skip evidence. A model sits on the left as a proposer whose quote is verified against the source, and on the right as an auditor returning supported / unsupported / unknown — in neither role can it originate a cell.

Technologies used in the solution

Python 3.12, standard library only. The pipeline imports nothing outside the stdlib — CSV and XLSX are read and written with zipfile and XML directly. There is no pandas, no openpyxl, no ML framework and no dependency that can break a build.

Core pipeline
Python 3.12 · standard library only · no packages

Data I/O
csv, zipfile, xml.etree — XLSX written by hand

Prototype UI
Gradio 6 on Hugging Face Spaces (the one dependency)

Optional model layer
Provider-neutral adapter: Groq, Gemini, Anthropic, OpenAI

Model governance
Evidence firewall, redundancy guard, verdict-only auditing

Quality assurance
46 invariant tests; evaluation against the published answer key

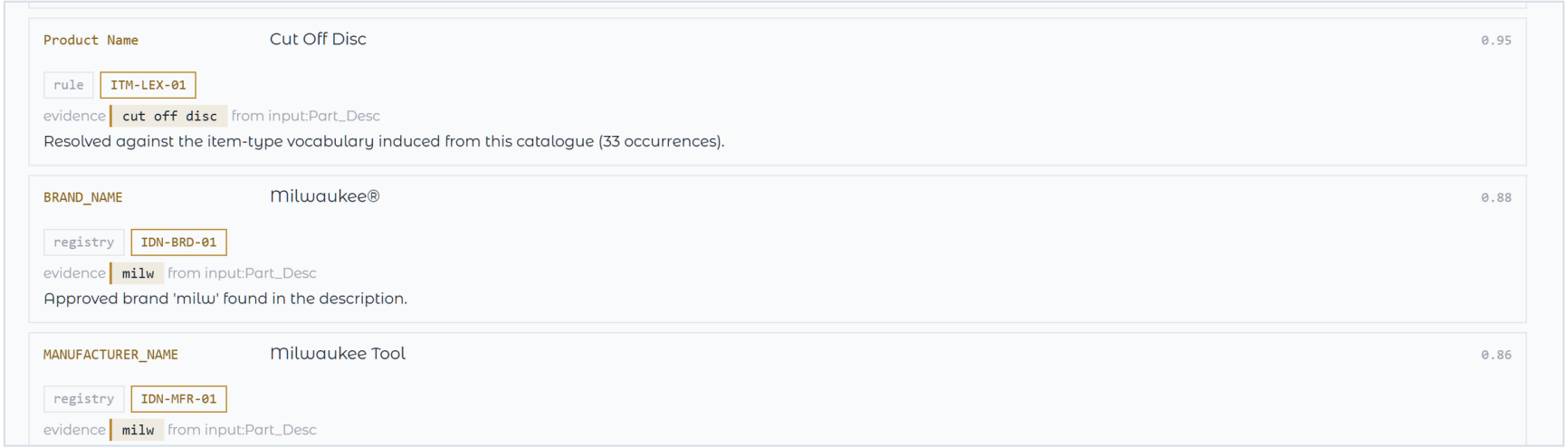
Consequence: a judge can clone the repository and run the whole pipeline with one command, on a machine with no network access.

Estimated implementation cost (optional)

Running the deterministic pipeline costs nothing but CPU.

Deterministic enrichment	**₹0**	No API calls. ~3 s per 1,000 rows on one core; ~1 CPU-hour per 1M SKUs.
Hosting the prototype	**₹0**	Hugging Face Spaces free tier.
Optional model pass	**≈ ₹40–90 per 10,000 rows**	Only rows the rules cannot resolve are sent — roughly 12% of the catalogue — and results are cached on disk.
Human review	**Scales with the queue**	77.4% of rows need no human. The queue is ranked, so reviewer time goes to the rows where it pays.

Snapshots of the MVP



Three columns of one row, as the prototype explains them. Each shows the method (rule, registry), the rule id, a confidence, and in the gold quote the exact characters of the input that justified the value — with the column they came from. BRAND_NAME became Milwaukee® because the four characters “milw” appear in Part_Desc and resolve against the approved brand registry; nothing here was generated.

26,596 populated cells across the catalogue carry provenance of this kind, produced by the pipeline rather than written afterwards as a report. Reproduce this panel by clicking any row at huggingface.co/spaces/jacklachan/unihack.

Additional Details/Future Development (if any)

What the measurements told us to build next.

Manufacturer retrieval is the real ceiling. Eleven of the twelve attribute values in the labelled row appear nowhere in its input. No amount of prompting recovers them — they have to be fetched from manufacturer sources and then held to the same evidence rule as everything else. This is the single highest-value extension.

A larger answer key. Accuracy is 48.0% exact on two labelled rows. That base is too narrow to carry a claim, which is why we report sibling agreement alongside it. With a few hundred labelled rows the same harness reports a real number.

Learning from corrections. Every reviewer edit is already captured with the fact it overrode. Feeding those back as registry and vocabulary entries makes the deterministic path absorb the reviewer's judgement permanently.

Image and document evidence. The Fact/Evidence type does not care whether a quote came from text or from a spec sheet. Extending the evidence kind lets datasheets and images become first-class sources without weakening the rule.

Provide links to your:

1. GitHub Public Repository
2. Demo Video Link (3 Minutes)
3. Working Prototype Link

Working prototype

https://huggingface.co/spaces/jacklachan/unihack

Opens already enriched. No login, no key, no setup.

GitHub repository

https://github.com/jacklachan/unihack

Standard library only, 46 invariant tests, clone and run.

Demo video (3 minutes)

<<< PASTE YOUR VIDEO LINK >>>

Set sharing to anyone-with-the-link and check it in a private window.

unilog | H2S HACK2SKILL

UniHack

AI-Powered Product Intelligence for
Industrial Commerce

Thank You